# ARTIFICIAL INTELLIGENCE

## Unit-II: Advanced Search

# Unit-II: Advanced Search

- Constraint Satisfaction (Backtracking, Local Search)
- Constructing Search Trees
- Stochastic Search
- A* Search Implementation
- Minimax Search
- Alpha-Beta Pruning

# Constraint Satisfaction Problem (CSP)

- states evaluated by heuristics and goal test conform to a **standard**, **structured** and simple representation

- **Constraint satisfaction problems** or **CSP**s are mathematical problems where one must **find** states or objects that **satisfy** a number of *constraints* or criteria.
- A constraint is a **restriction** of the **feasible solutions** in an optimization problem.

# Constraint Satisfaction Problem (CSP)

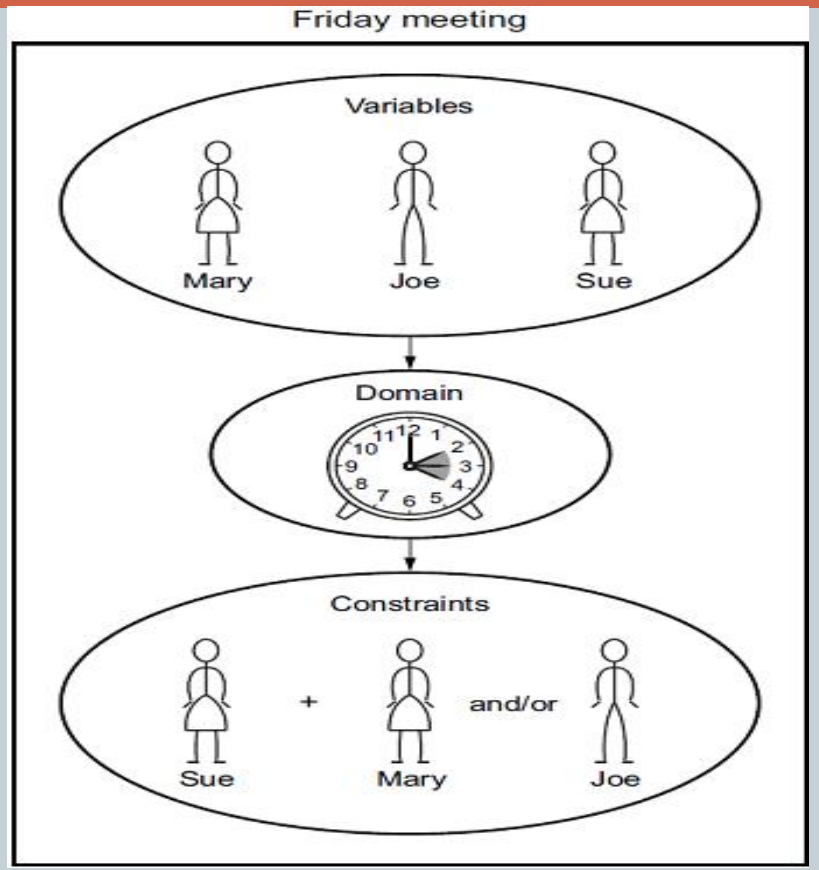
- CSP is a problem composed of a
  - finite set of **variables**,
  - each of which has a finite **domain** of **values**, and
  - a set of **constraints**
- Each constraint is defined over some subset of the original **set** of **variables** and **restricts** the values these variables can simultaneously take.
- The task is to find an **assignment** of a value for each variable such that the assignments **satisfy** all the constraints.

# Examples

## Example 1

- Different classrooms are of different capacity and an examination can only be scheduled in a classroom that has enough seats for students who are going to take that examination
- Some students may take part in several examinations and these examinations can not be scheduled in the same time slot.

## Example 2



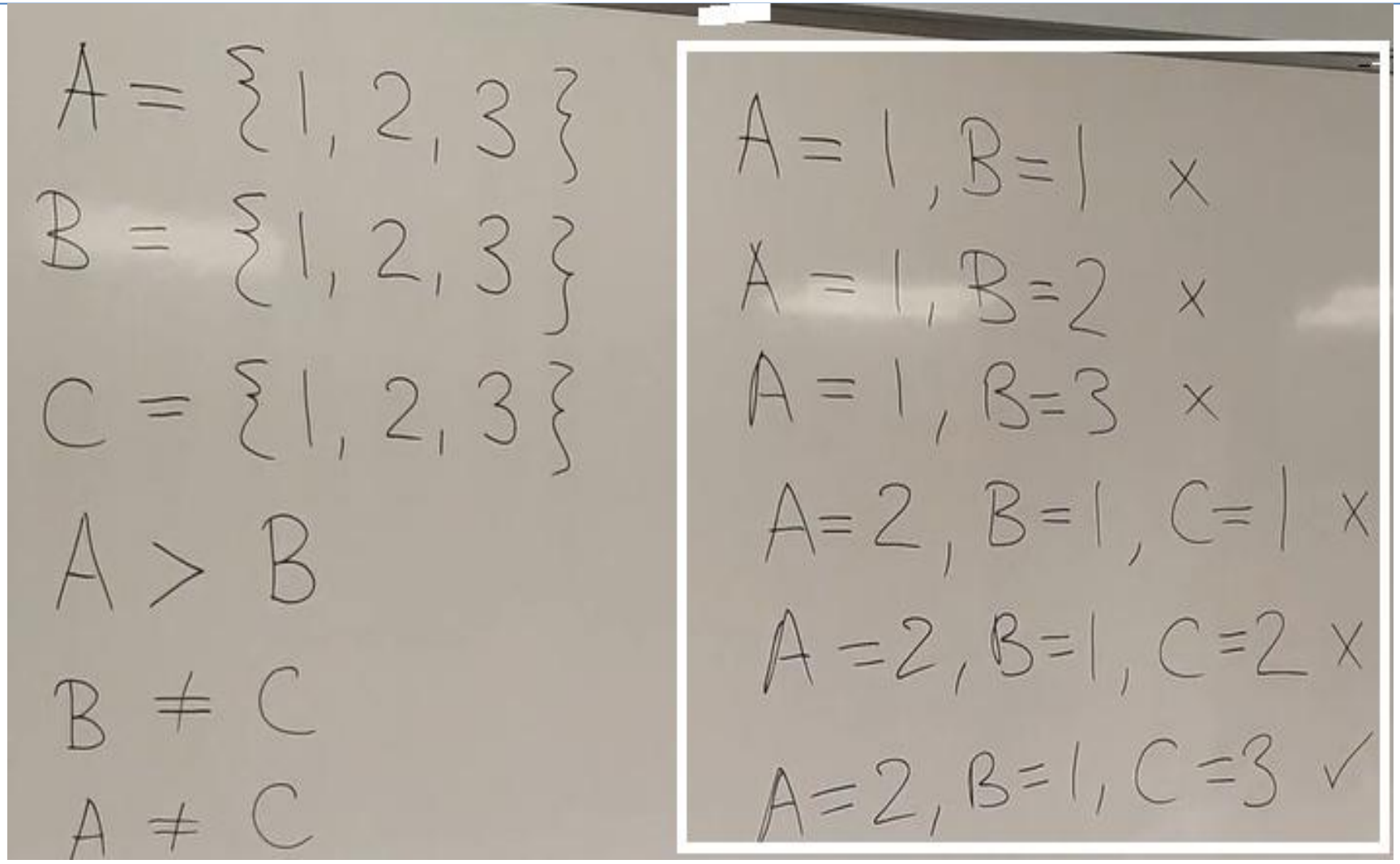
# Constraint satisfaction problems

- What is a CSP?
  - **Finite set of variables $V_1$ , $V_2$ , …, $V_n$**
  - **Finite set of constrains $C_1$, $C_2$, …, $C_m$**
  - **Nonemtpy domain of possible values for each variable $D_{V1}$, $D_{V2}$, … $D_{Vn}$**
  - **Each constraint $C_i$ limits the values that variables can take, e.g., $V_1 \neq V_2$**
- A *state* is defined as an *assignment* of values to some or all variables.
- *Consistent assignment*: assignment does not violate the constraints.

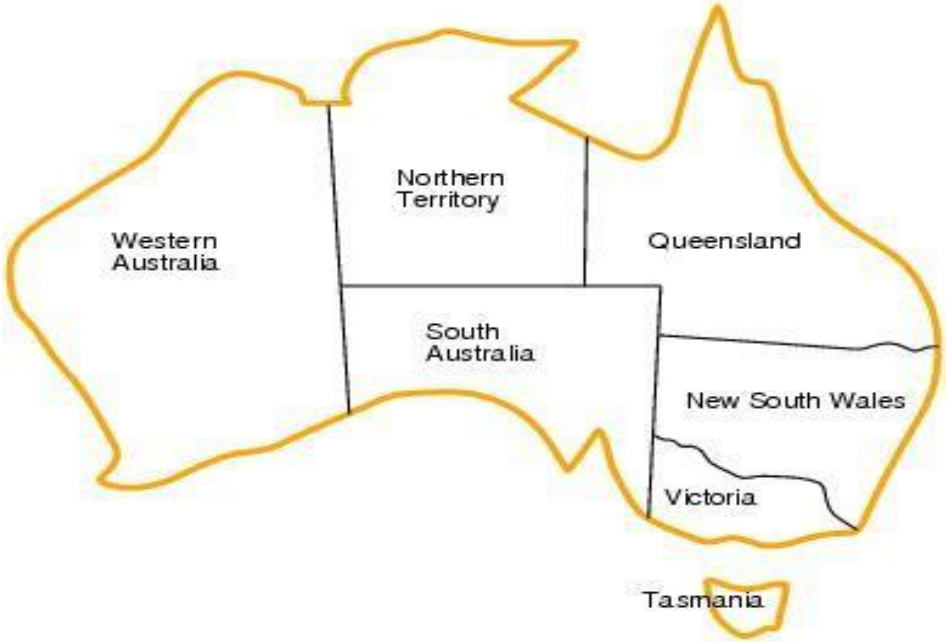
# Constraint satisfaction problems

- An assignment is *complete* when every value is mentioned.
- A *solution* to a CSP is a complete assignment that satisfies all constraints.
- Some CSPs require a solution that maximizes an *objective function*.
- Applications: Scheduling the time of observations on the Hubble Space Telescope, Floor planning, Map coloring, Cryptography

# Constraint satisfaction problems

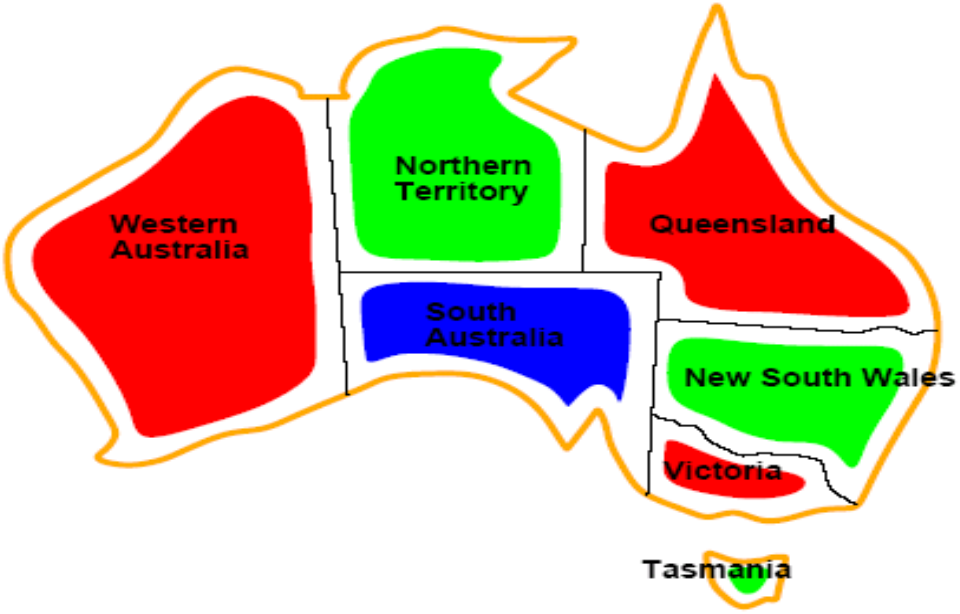


# CSP example: map coloring




- Variables: *WA, NT, Q, NSW, V, SA, T*
- Domains: $D_i$=*{red,green,blue}*
- Constraints:adjacent regions must have different colors.
  - E.g. $WA \neq NT$ (if the language allows this)
  - E.g. *(WA,NT) = {(red,green),(red,blue),(green,red),…}*

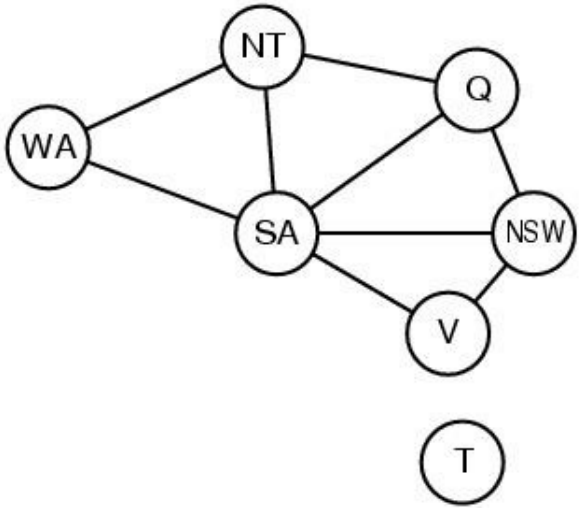
# CSP example: map coloring




- Solutions are assignments satisfying all constraints, e.g.

  $\{WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green\}$

# Constraint graph




- CSP benefits
  - **Standard representation pattern**
  - **Generic goal and successor functions**
  - **Generic heuristics (no domain specific expertise).**

- Constraint graph = nodes are variables, edges show constraints.
  - **Graph can be used to simplify search.**
    - e.g. Tasmania is an independent subproblem.

# Backtracking Search

## aka Brute-Force Search

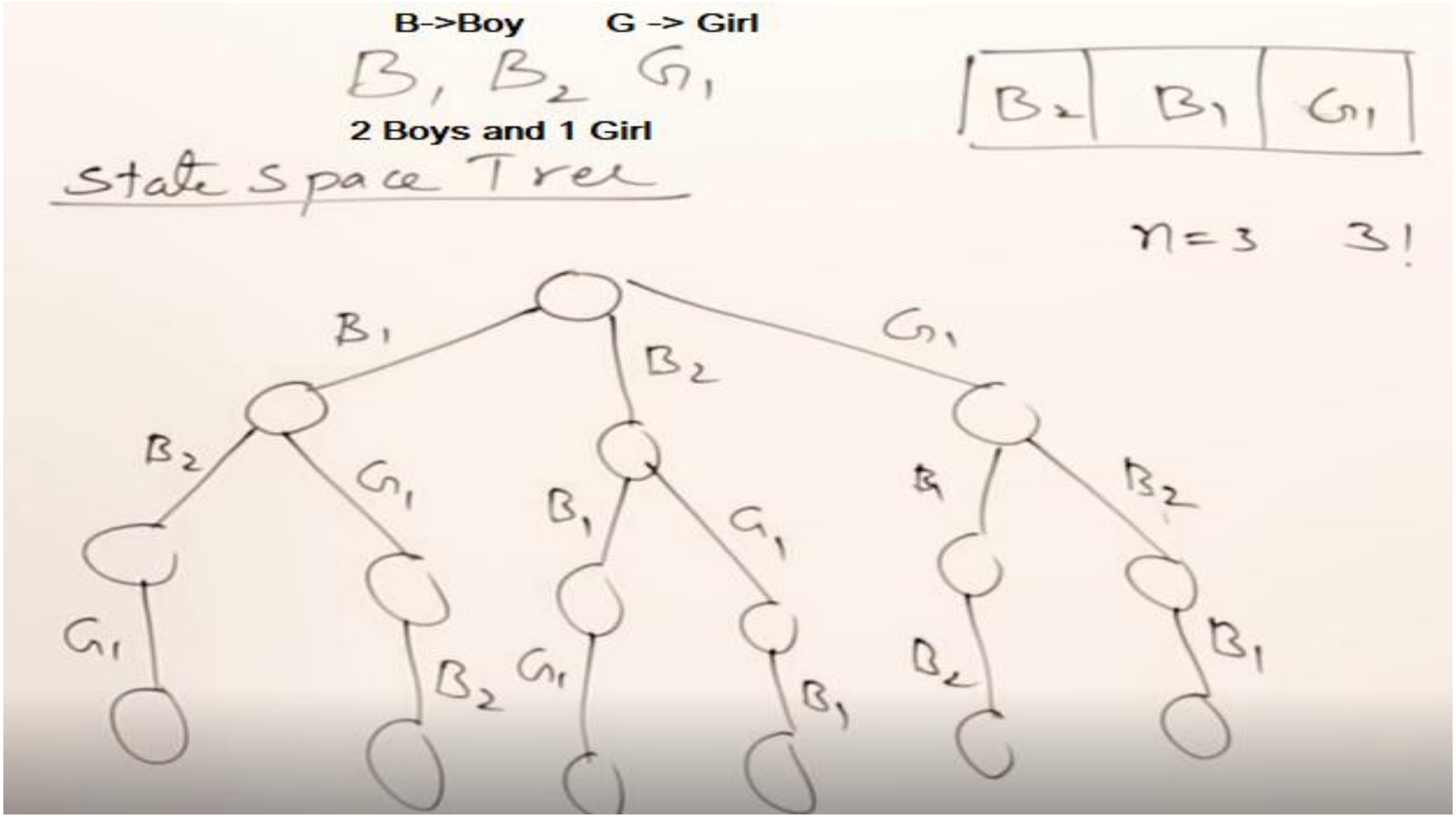
- It is similar to Depth-first search
- Chooses values for **one variable** at a **time** and backtracks when a **variable** has no legal values left to assign.
- Uninformed algorithm
  - **No good general performance (see table p. 143)**

# Backtracking search

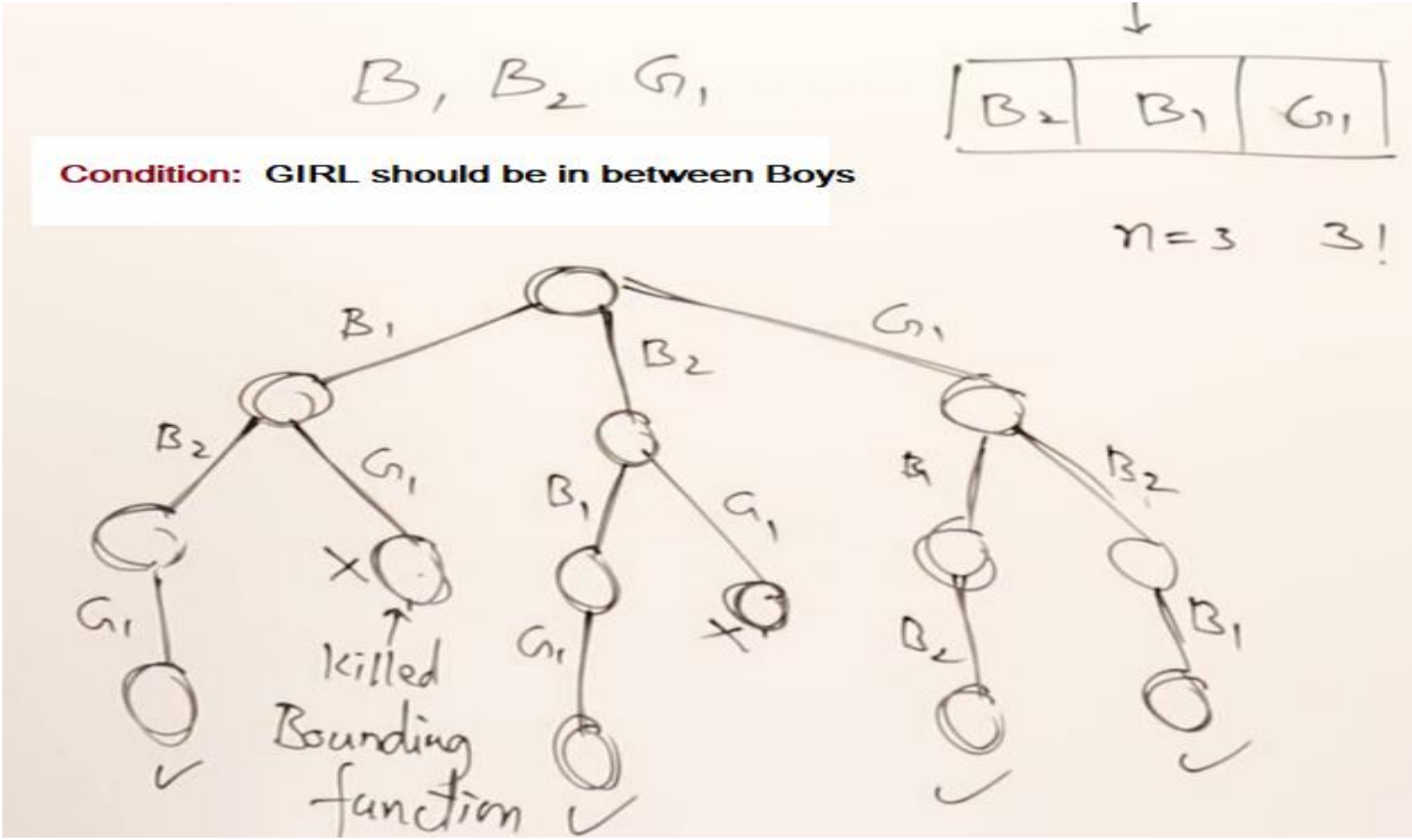
**function** BACKTRACKING-SEARCH(*csp*) **return** a solution or failure
    **return** RECURSIVE-BACKTRACKING(*{} , csp*)

**function** RECURSIVE-BACKTRACKING(*assignment, csp*) **return** a solution or failure
    **if** *assignment* is complete **then return** *assignment*
    *var* ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[*csp*],*assignment*,*csp*)
    **for each** *value* **in** ORDER-DOMAIN-VALUES(*var, assignment, csp*) **do**
        **if** *value* is consistent with *assignment* according to CONSTRAINTS[*csp*] **then**
            add *{var=value}* to assignment
            *result* ← RRECURSIVE-BACTRACKING(*assignment, csp*)
            **if** *result* ≠*failure* **then return** *result*
            remove *{var=value}* from *assignment*
    return *failure*

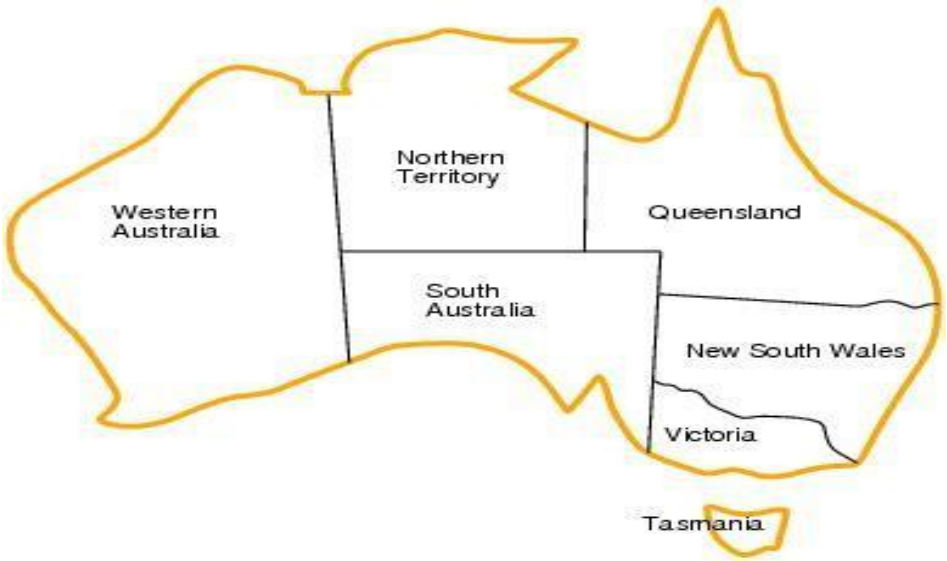
# Backtracking search



# Backtracking search



# Backtracking example




Variables: *WA, NT, Q, NSW, V, SA, T*

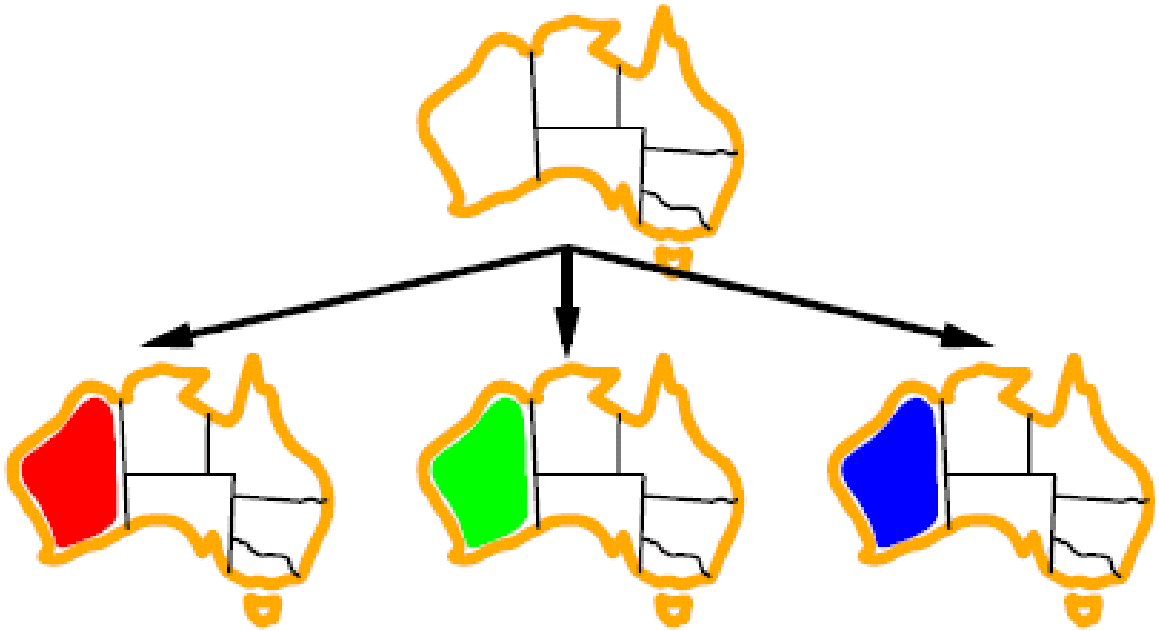
Domains: $D_i$=*{red,green,blue}*

Constraints:adjacent regions must have different colors.

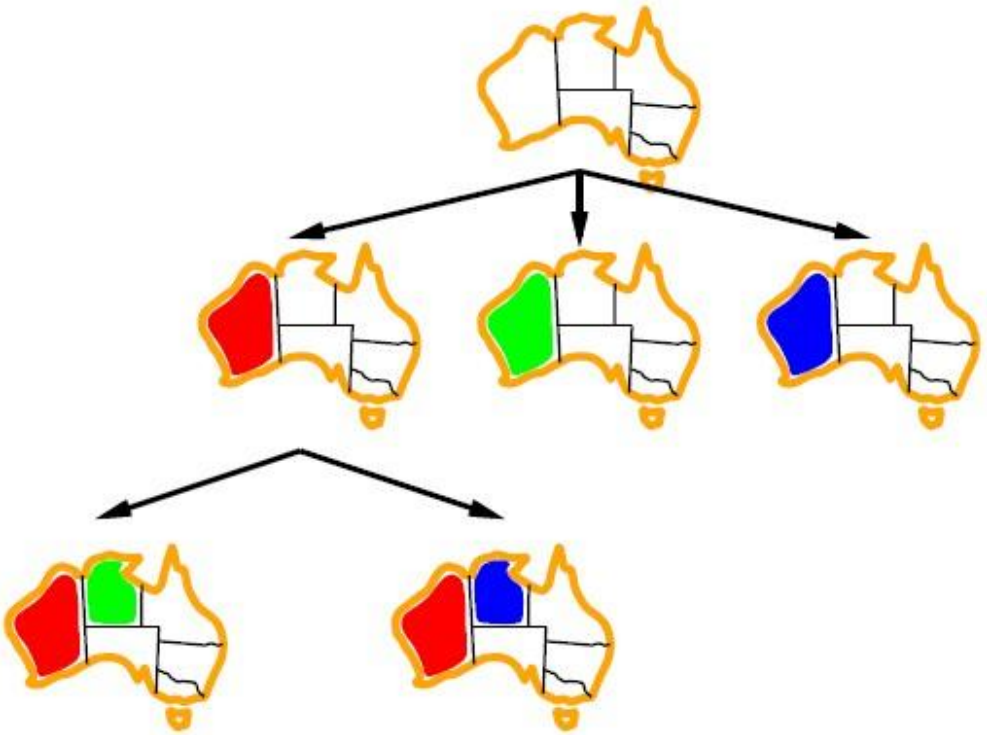
E.g. $WA \neq NT$ (if the language allows this)

E.g. *(WA,NT) = {(red,green),(red,blue),(green,red),...}*

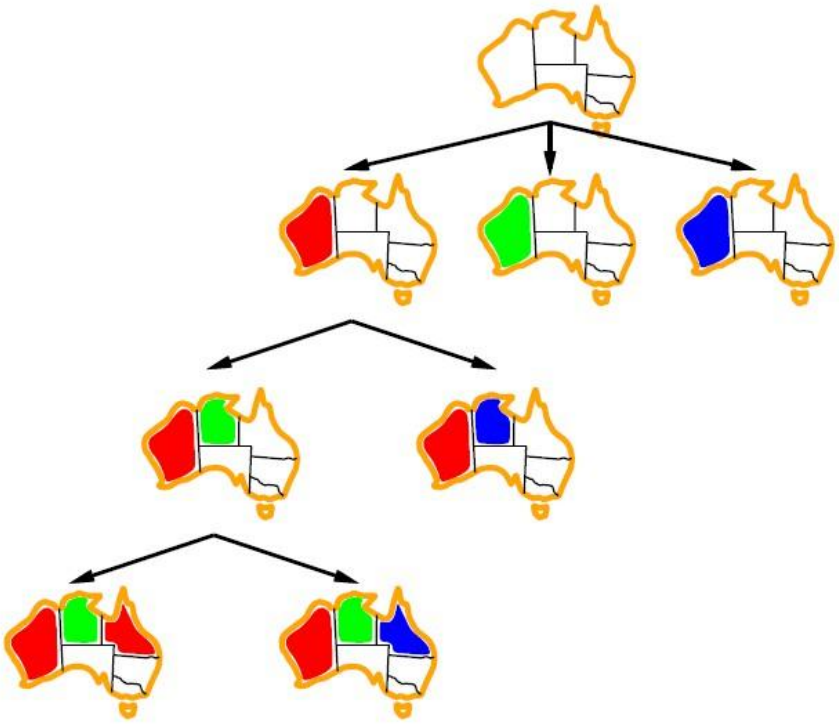
# Backtracking example



# Backtracking example



# Backtracking example



# Improving backtracking efficiency

- Previous improvements → introduce heuristics
- General-purpose methods can give huge gains in speed:
  - **Which variable should be assigned next?**
  - **In what order should its values be tried?**
  - **Can we detect inevitable failure early?**
  - **Can we take advantage of problem structure?**

# Local Search

- They apply mostly to problems for which we **don't** need to know the **path** to the solution but only the **solution** itself.
- They operate using a single state or a **small number of states** and explore the neighbors of that state. They usually **don't store** the path.
- A particular case are optimization problems for which we **search** for the **best solution** according to an **objective** function.
- The **distribution** of **values** of the objective function in the state space is called a **landscape**.

# Local Beam Search

- In this algorithm, it holds **k number** of states at any given time. At the start, these states are generated randomly.
- The successors of these k states are computed with the help of objective function.
- If any of these successors is the maximum value of the objective function, then the algorithm stops.
- Otherwise the (initial k states and k number of successors of the states = 2k) states are placed in a pool.

**The pool is then sorted numerically. The highest k states are selected as New initial states. This process continues until a maximum value is reached.**

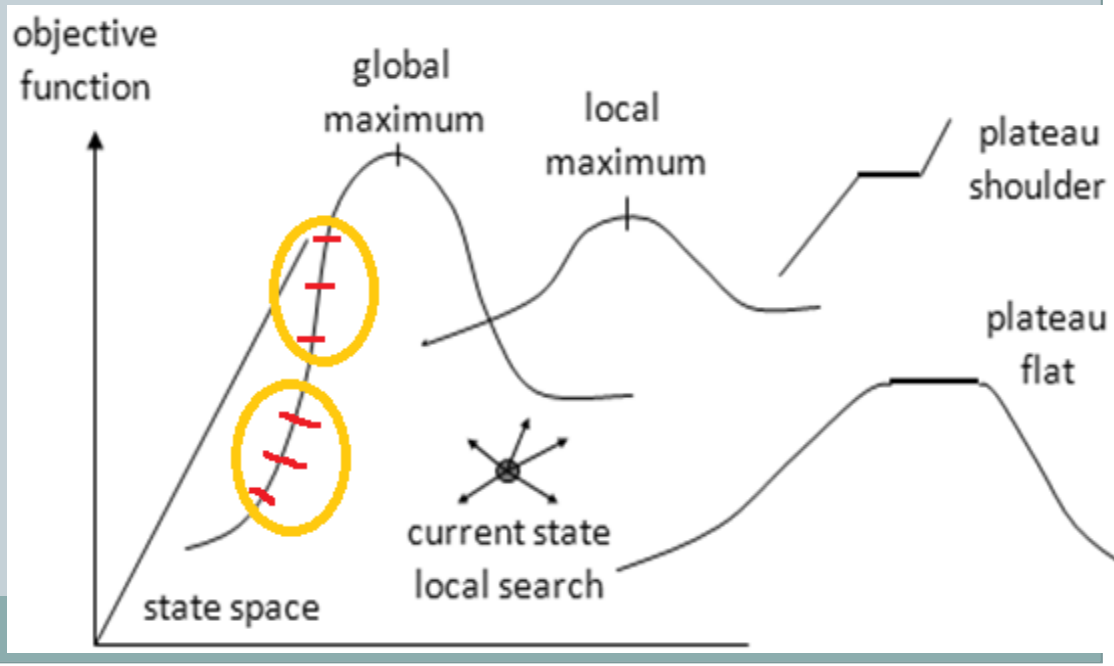
## Local Beam Search

Travelling Salesman Problem


Keeps track of k states rather than just 1. k=2 in this example. Start with k randomly generated states.

# Local Search

- Global maximum - a state that maximizes the objective function over the entire landscape.
- Local maximum - a state that maximizes the objective function in a small area around it.
- Plateau - a state such that the objective function is constant in an area around it.
- Shoulder - a plateau that has an uphill edge.

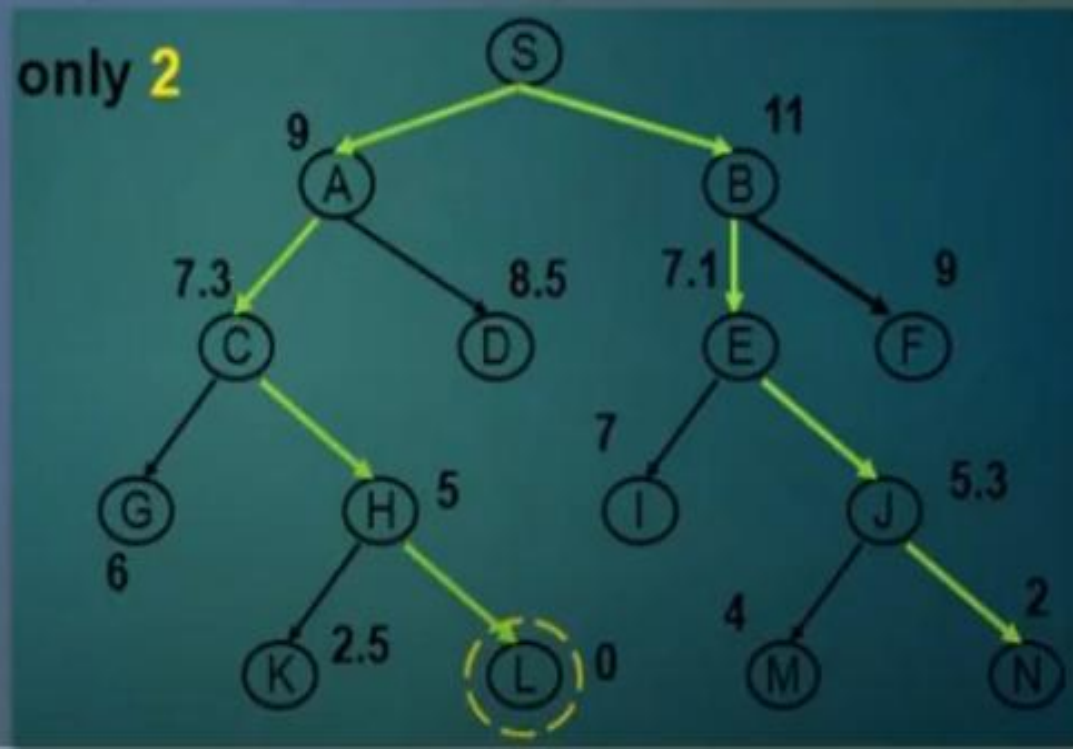


# Local Search

## Local Beam Search

- Keeping just one node in memory might seem to be an extreme reaction to the problem of memory limitations. The **local beam search** algorithm keeps track of k states rather than just one.
- It begins with k generated states. At each step, all the successors of all k states are generated. If any one is a goal, the algorithm halts. Otherwise, it selects the k best successors from the complete list and repeats.

| Frontier List | Explored |
|---|---|
| A, B | S |
| C, E | S , A, B |
| H, J | S, A, B, C, E |
| L, N | S, A, B, C, E, H, J |
| | S, A, B, C, E, H, J, L, N |



# Constructing Search Trees

- **Search:** Searching is a step by step procedure to solve a search-problem in a given search space.
- **Search tree:** A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.

A search problem consists of:

- **A State Space.** Set of all possible states where you can be.
- **A Start State.** The state from where the search begins.
- **A Goal Test.** A function that looks at the current state returns whether or not it is the goal state.

# Constructing Search Trees

- Create search trees to solve Artificial Intelligence questions.
- Use each piece of information as a **node** and construct a **hierarchy** of nodes based on how the individual pieces of information **connect** together so that one can go along a path through a series of nodes in order to get from one to another.

- **The search tree will find a potential solution to unknown questions by looking at the available data in an organized manner, one piece (or "node") at a time.**

# Constructing Search Trees

- Suppose one is writing a program for an airline to find a way for a customer to get from City A to City B given a list of all of the possible flights between any two cities. The program will need to go through the list and determine which flight (or connecting series of flights) that the customer should take. This is implemented using a search tree.
- City A (where the customer currently is located) will be the initial piece of information, or "initial node." The search tree will consist of this initial node at the top of the tree.
- Next, the program would look at all of the flights beginning with City A. These flights would show cities that the customer could reach in just one flight, and so these cities (each city a node itself) are on the next level, below the initial node.
- Now, each of these cities has flights as well, which would be on the third level, and the tree would continue so forth.

# Constructing Search Trees

- Choosing a search strategy (i.e. Breadth-first or Depth-first) will tell the program how to progress down the search tree, or hierarchy of nodes that has been built from the information. There are clearly many different ways that one can get from City A to City B, so the search strategy is the key to determining which path will be selected.
- Search trees, with search strategies, can also be found in finding solutions in puzzle-solving or simple games such as "The Eight Queens Problem" where one tries to place eight queens in locations on a chessboard such that none of them can move to the space of another.

# Stochastic Search

- Stochastic search is one of the hill climbing search algorithms, that does not examine for all its **neighbor** before moving.
- Rather, this search algorithm selects one neighbor node at **random** and decides whether to **choose** it as a current state or **examine** another state.
- Under a stochastic beam search it is possible for an **individual** to be selected multiple times at **random**.

# Stochastic Search

- Stochastic pretty much means **randomized** in some way.
- One of the major issues with beam search is that it tends to get **stuck** into **local optima** instead of the global optimum.
- To avoid that stochastic search gives some(most often small) probability of the solution to choose the step that is not optimal at a given moment.

# Stochastic Beam Search

- In the Stochastic beam search instead of choosing the best k individuals, it selects k number of the individuals at **random**; the individuals with a better evaluation are more likely to be chosen. This is done by making the **probability** of being chosen as a function of the evaluation function.
- Stochastic beam search tends to allow more **diversity** in the k individuals than does plain beam search.
- In terms of evolution in biology, the evaluation function reflects the fitness of the individual; the fitter the individual, the more likely it is to pass that part of its variable task that is good over the next generation.
- Stochastic beam search is similar to asexual reproduction within which every individual offers slightly mutated children and then stochastic beam search proceeds with the survival of the fittest.

# Mini-max Search

- Mini-max algorithm is a **recursive** or backtracking algorithm which is used in decision-making and game theory.
- It provides an **optimal move** for the player assuming that opponent is also playing optimally.
- Mini-Max algorithm uses recursion to search through the game-tree.
- Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various tow-players game. This Algorithm computes the minimax decision for the current state.

# Mini-max Search

- In this algorithm two players play the game, one is called **MAX** and other is called **MIN**. Both the players fight it as the opponent player gets the minimum **benefit** while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a **depth-first search** algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

# Mini-max Search: Psedo Code

```
function minimax(node, depth, maximizingPlayer) is
if depth ==0 or node is a terminal node then
return static evaluation of node
if MaximizingPlayer then      // for Maximizer Player
maxEva= -infinity
 for each child of node do
 eva= minimax(child, depth-1, false)
maxEva= max(maxEva,eva)       //gives Maximum of the values
return maxEva
 else                         // for Minimizer player
 minEva= +infinity
 for each child of node do
 eva= minimax(child, depth-1, true)
 minEva= min(minEva, eva)       //gives minimum of the values
 return minEva
```

**Ques:-** Explain MinMax theorem. [Game Playing]

It is a specialized Search Algo that returns optimal sequence of moves for a player in Zero Sum Game.

↳ Recursive/Backtracking algo which is used in decision making and Game theory. → Two Player.

↳ uses recursion to search through Game Tree.

↳ Algo Computes minimax decision for current State.

↳ Two Players → MAX [Selects maximum Value]
→ MIN [Selects minimum Value]

↳ Depth-first Search Algo is used for exploration of Complete Game Tree.

↳ CHESS, Checkers, Tic-tac-toe...

[max = $-\infty$] Worst Values.
[min = $\infty$] [Initial]

**Properties:-** → COMPLETE

① Definitely found sol$^n$ (if exists)

② optimal

③ Time Complexity = $O(b^m)$ (m → maximum depth.)
↳ branching factor of game tree

④ Space Complexity = $O(b^m)$

**Limitations:-**

↳ Slow for complex Games such as chess.

↳ 35 choices/moves.

$d$ = 100 (for both players)

$(35)^{100}$ ↳ BIG.

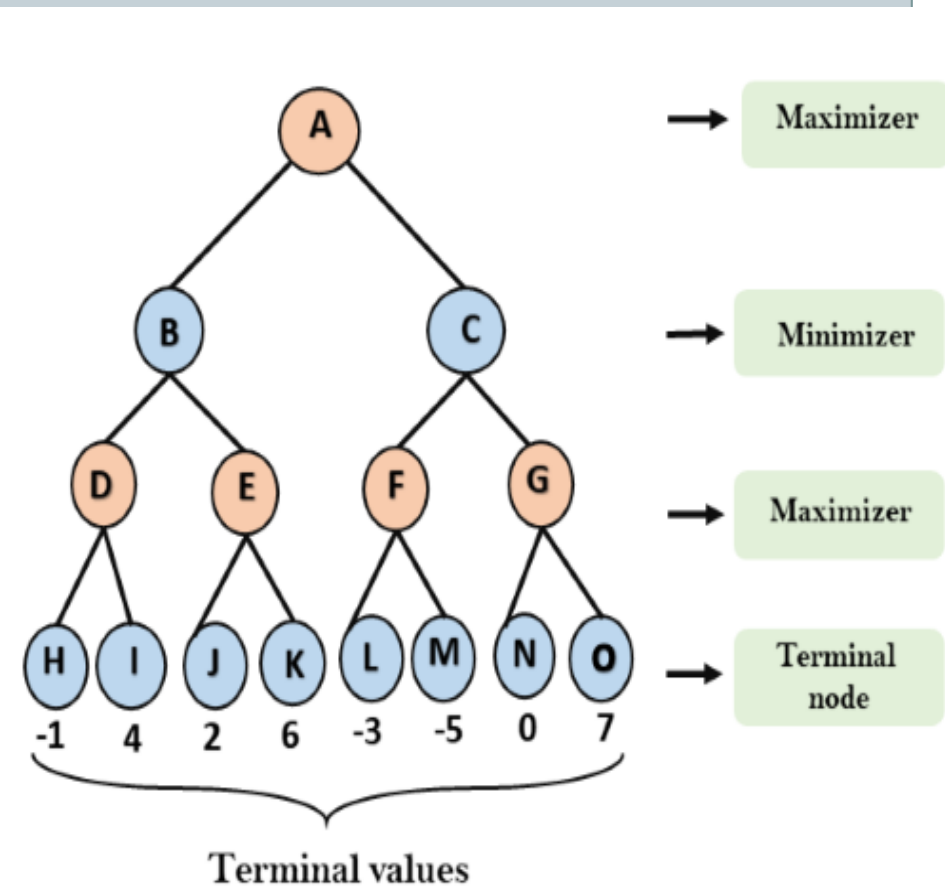
# Working of Mini-max Search

- The working of the minimax algorithm can be easily described using an example. Below we have taken an example of game-tree which is representing the two-player game.
- In this example, there are two players one is called Maximizer and other is called Minimizer.
- Maximizer will try to get the Maximum possible score, and Minimizer will try to get the minimum possible score.
- This algorithm applies DFS, so in this game-tree, we have to go all the way through the leaves to reach the terminal nodes.
- At the terminal node, the terminal values are given so we will compare those value and backtrack the tree until the initial state occurs. Following are the main steps involved in solving the two-player game tree:

# Working of Mini-max Search

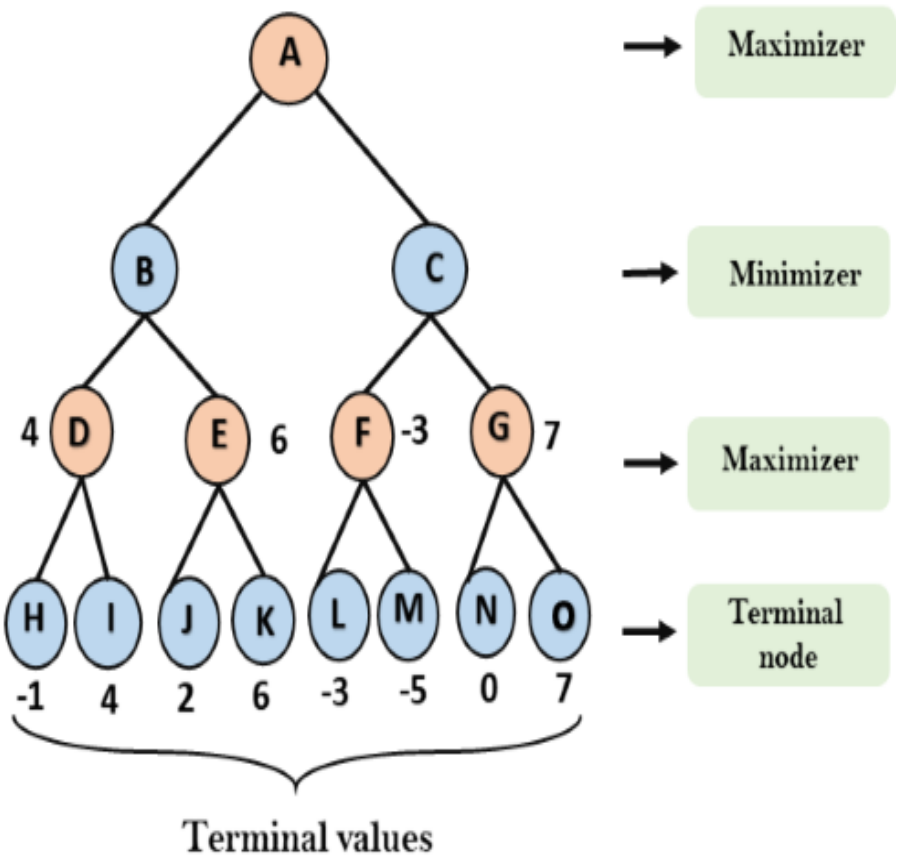
- **Step-1:** In the first step, the algorithm generates the entire game-tree and apply the utility function to get the utility values for the terminal states.
- In the below tree diagram, let's take A is the initial state of the tree.
- Suppose maximizer takes first turn which has worst-case initial value =- infinity,

and minimizer will take next turn which has worst-case initial value = +infinity



# Working of Mini-max Search

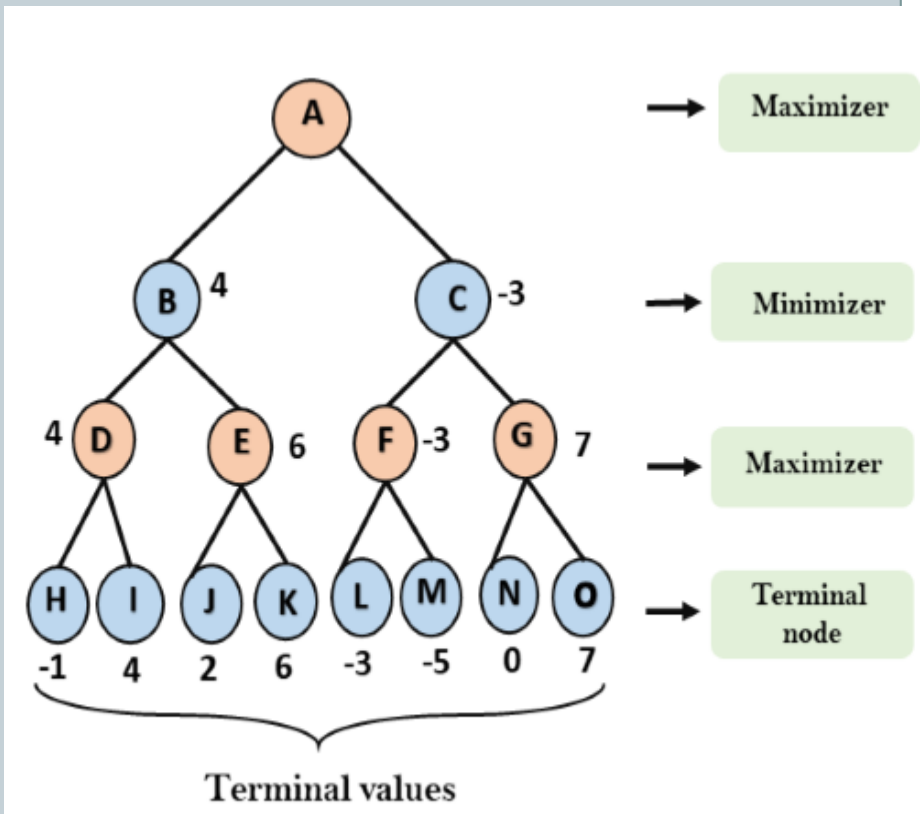
- **Step 2:** Now, first we find the utilities value for the Maximizer, its initial value is -∞, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values. It will find the maximum among the all.
- For node D max(-1,- -∞) => max(-1,4)= 4
- For Node E max(2, -∞) => max(2, 6)= 6
- For Node F max(-3, -∞) => max(-3,-5) = -3
- For node G max(0, -∞) = max(0, 7) = 7



# Working of Mini-max Search

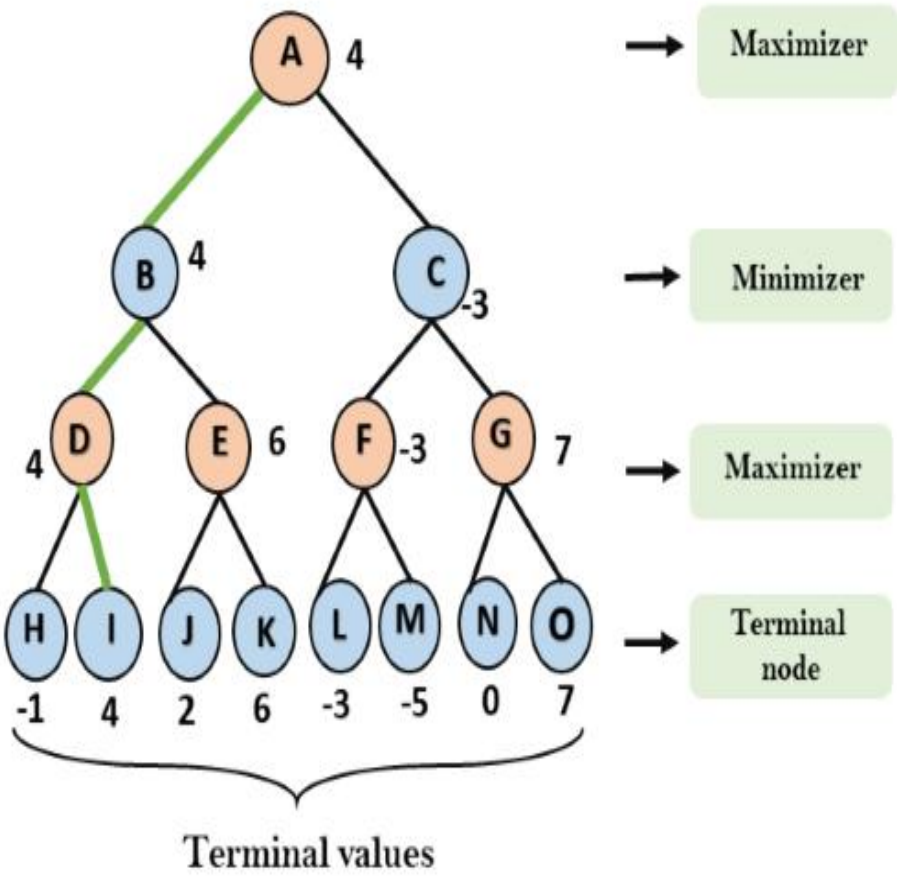
- **Step 3:** In the next step, it's a turn for minimizer, so it will compare all nodes value with +∞, and will find the 3rd layer node values.

- For node B= min(4,6) = 4
- For node C= min (-3, 7) = -3



# Working of Mini-max Search

- **Step 4:** Now it's a turn for Maximizer, and it will again choose the maximum of all nodes value and find the maximum value for the root node.
- In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.
- For node A max(4, -3)= 4



# Limitation of the Mini-max Search

- The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc. This type of games has a huge branching factor, and the player has lots of choices to decide.
- This limitation of the minimax algorithm can be improved from **alpha-beta pruning** which we have discussed in the next topic.

# Alpha-Beta Pruning

- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half.
- Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**.
- This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.

# Alpha-Beta Pruning

- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.
- The two-parameter can be defined as: **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is **-∞**.
- **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is **+∞**.

# Alpha-Beta Pruning

- The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

# Condition for Alpha-Beta Pruning

The main condition which required for alpha-beta pruning is:

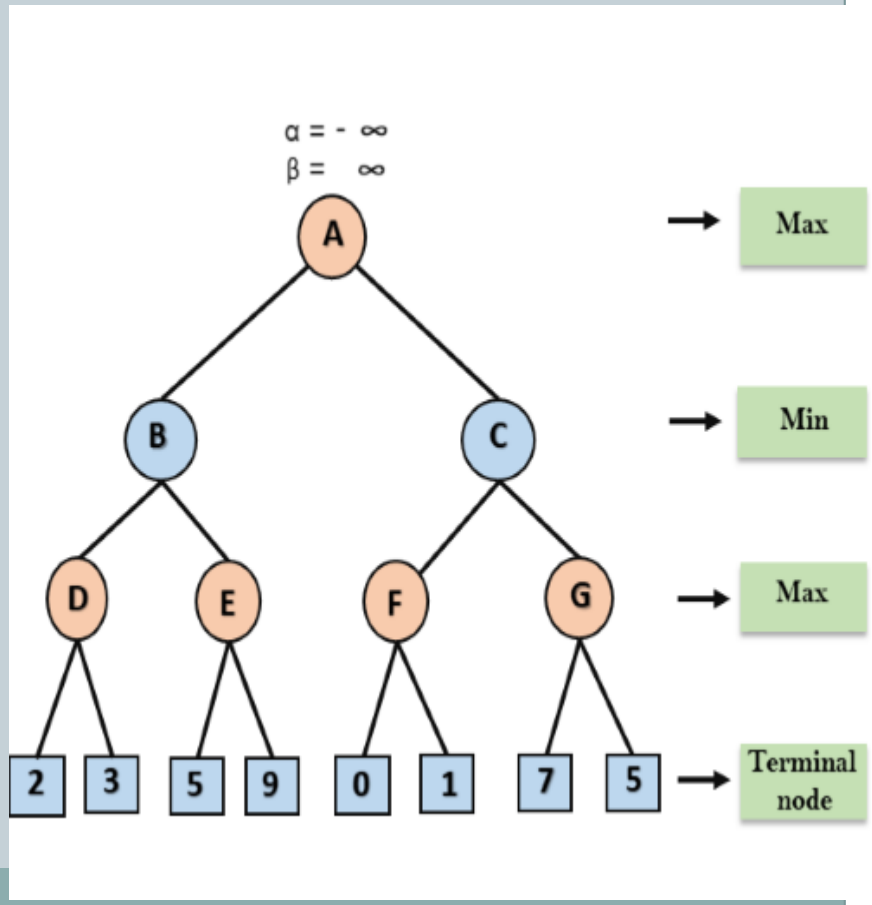
$$\alpha >= \beta$$

**Key points about alpha-beta pruning:**

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.

# Working of Alpha-Beta Pruning

- Let's take an example of two-player search tree to understand the working of Alpha-beta pruning
- **Step 1:** At the first step the, Max player will start first move from node A where α= -∞ and β= +∞, these value of alpha and beta passed down to node B where again α= -∞ and β= +∞, and Node B passes the same value to its child D.

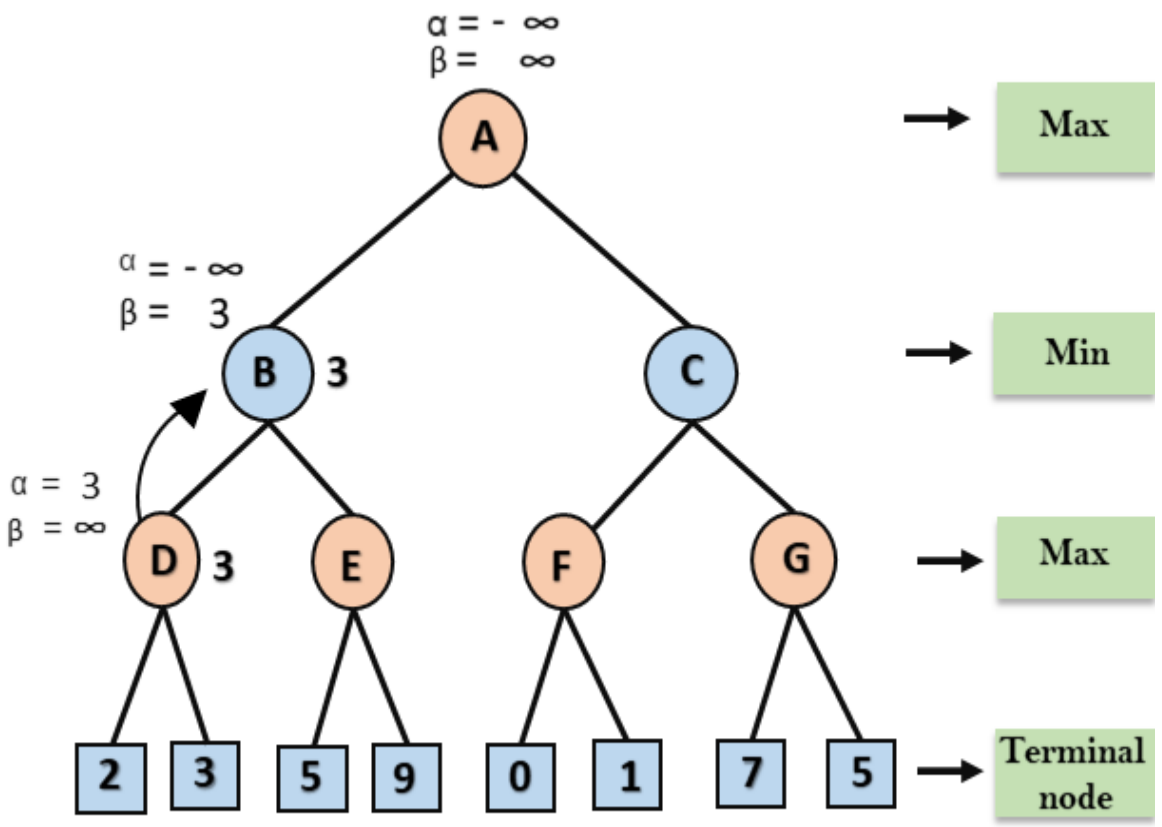


# Working of Alpha-Beta Pruning

- **Step 2:** At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.
- **Step 3:** Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3.
- In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.

# Working of Alpha-Beta Pruning

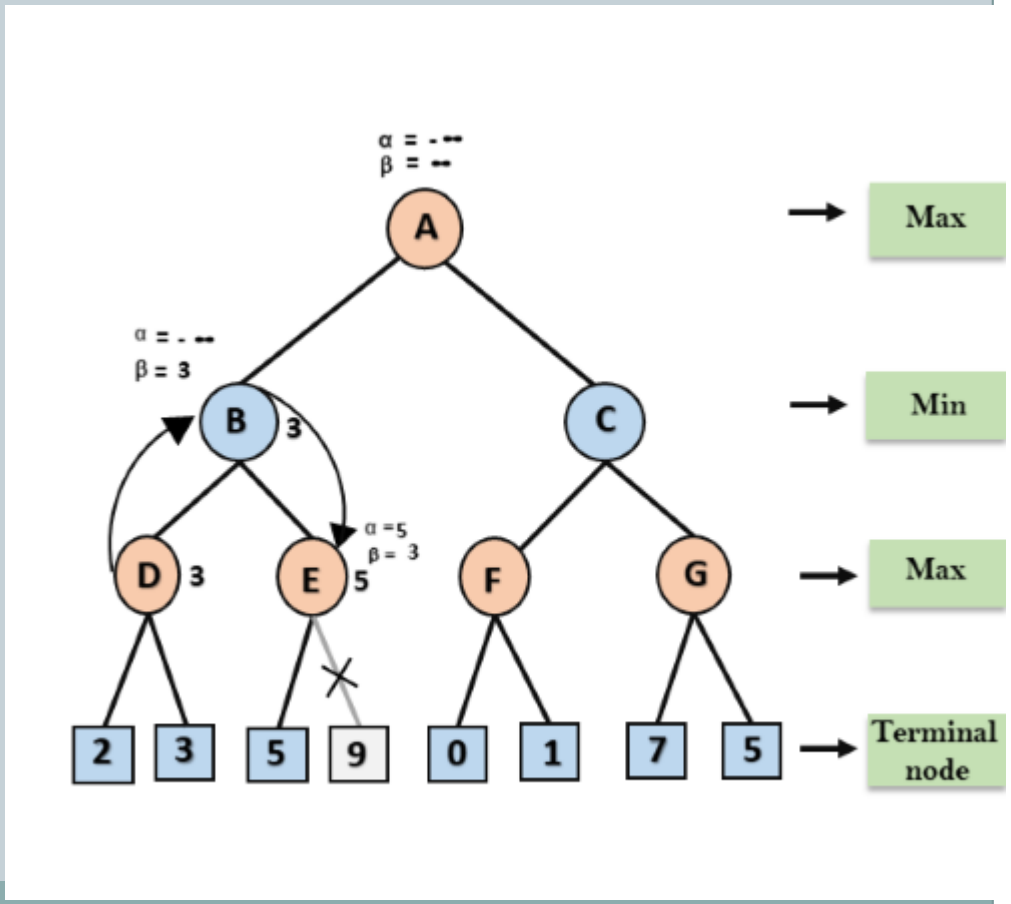
- **Step 4:** At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



# Working of Alpha-Beta Pruning

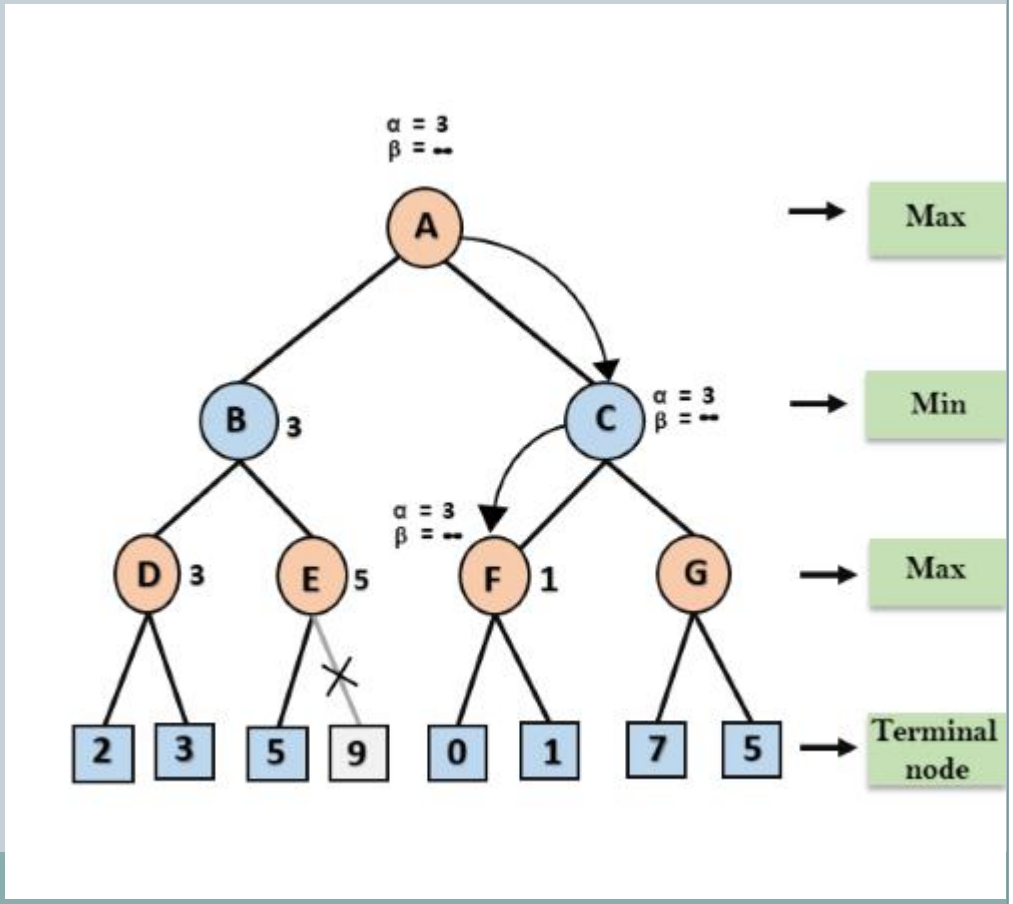
- **Step 5:** At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C.

- At node C, α=3 and β= +∞, and the same values will be passed on to node F.



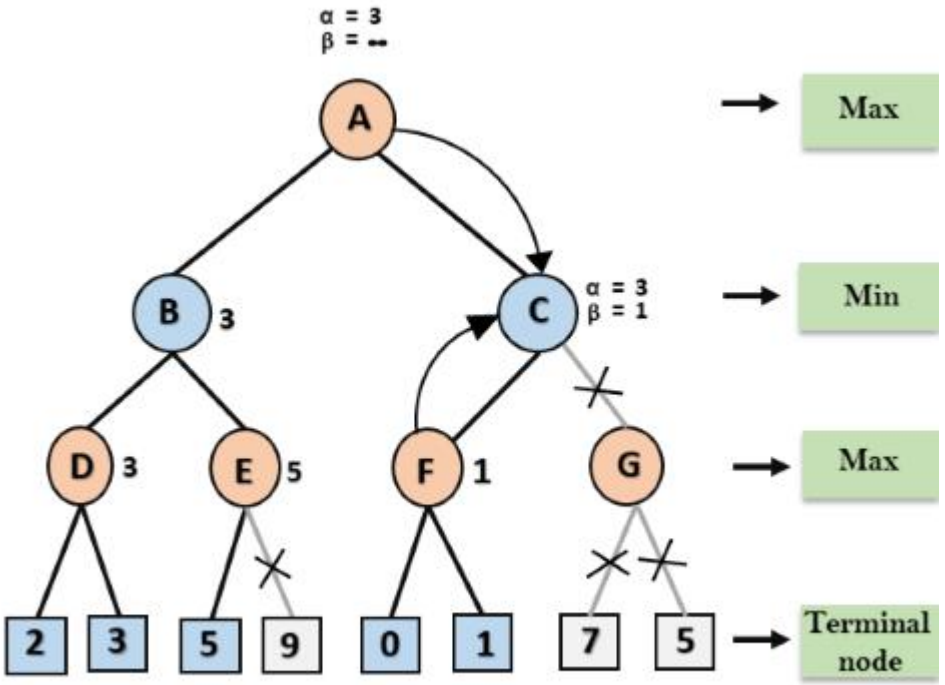
# Working of Alpha-Beta Pruning

- **Step 6:** At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.



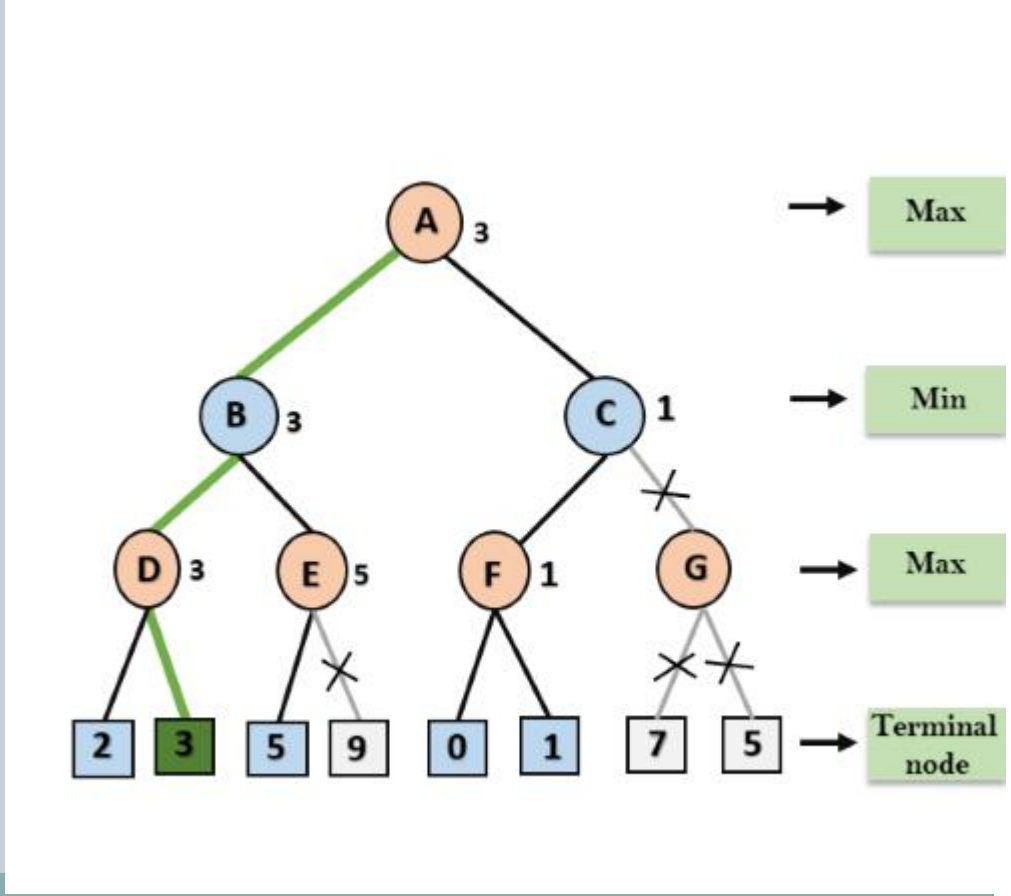
# Working of Alpha-Beta Pruning

- **Step 7:** Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



# Working of Alpha-Beta Pruning

- **Step 8:** C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.



- **Move Ordering in Alpha-Beta pruning:**
- The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning.
- It can be of two types:
- **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is $O(b^m)$.
- **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree. We apply DFS hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. Complexity in ideal ordering is $O(b^{m/2})$.